Proc Blart TUI

A Python process-monitoring antivirus for Windows: live process visibility, VirusTotal hash reputation checks, editable reaction policy, JSONL logs, and safe dry-run operation.

PROJECT TYPE	MAIN CODE	POLICY FILE
Process-monitoring antivirus	`procblart.py`	`policy.json`

1. Project Overview

Proc Blart TUI is a terminal-based security tool that continuously observes Windows processes, investigates the executable behind each process, checks known file hashes against VirusTotal, evaluates suspicious behavior against a local policy, and records the actions that would be taken or are actually taken.

The program runs safely in dry-run mode by default, so it can be used for observation and testing without killing, suspending, dumping, or quarantining processes. Execute mode is available for lab testing when the user intentionally starts the monitor with `-exec` .

> **Safety design:** remote monitoring is read-only, unknown files are not uploaded to VirusTotal by default, and VirusTotal lookups are rate-limited and cached.

2. Real-Time Process Monitor

The first requirement was to list active processes in real time and show process name, PID, CPU usage, and memory usage. Proc Blart implements this with `psutil.process_iter()` and refreshes a Rich-based terminal UI.

The live table displays:

- PID
- process name
- CPU percentage
- memory usage in MB
- VirusTotal status
- detection count
- policy match
- process start time
- executable path

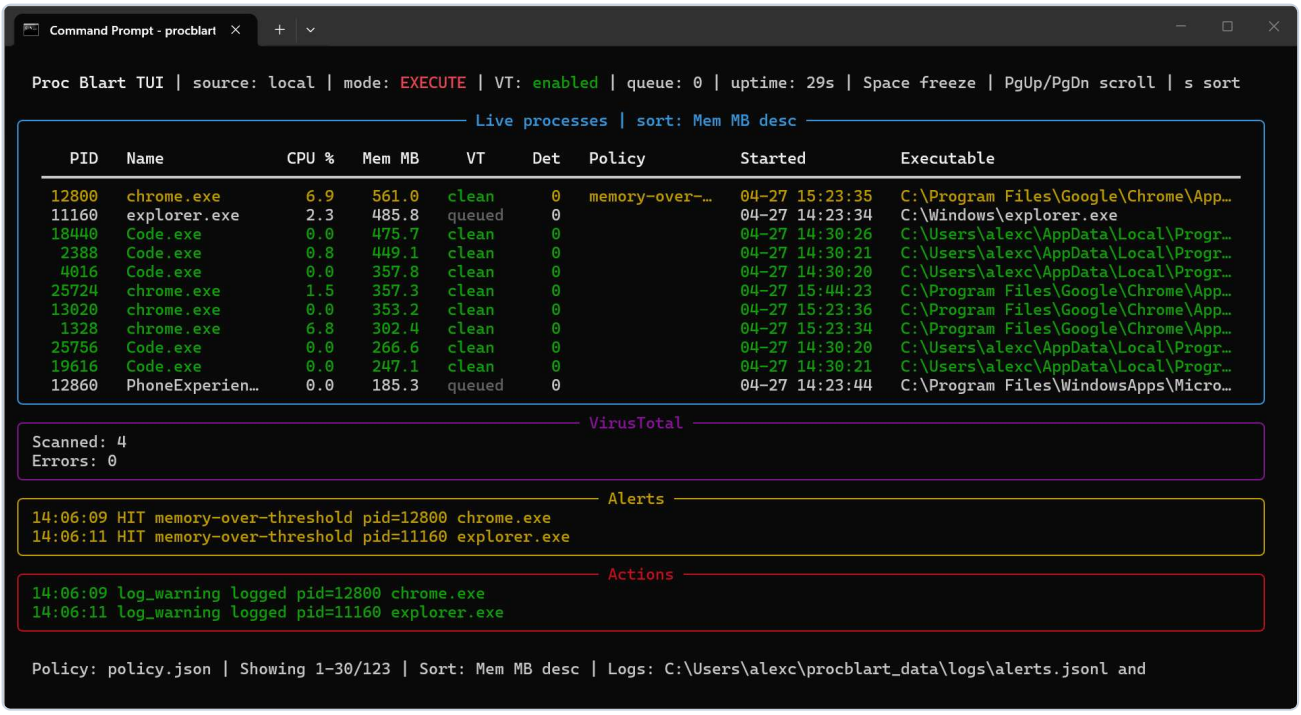

Figure 1: Proc Blart in a regular terminal window. The process table is readable, scrollable, sortable, and shows VirusTotal and policy status beside each process.

The TUI includes controls for freezing the view, scrolling through long process lists, sorting, reversing the sort order, and cleanly exiting the monitor.

Key	Function
`Space`	Freeze or resume the live display.
`PageUp` / `PageDown`	Scroll by one page.
`Up` / `Down`	Scroll by one row.
`Home` / `End`	Jump to the top or bottom of the process list.
`s`	Cycle sort order.
`r`	Reverse the current sort.
`q` / `Ctrl+X`	Exit cleanly.

3. VirusTotal Executable Checks

For each process, the monitor finds the executable path and calculates its SHA-256 hash. That hash is checked against the VirusTotal API v3. If VirusTotal reports malicious or suspicious detections, the process row is clearly highlighted in the interface.

Results are cached by SHA-256 in `procblart_data/cache/vt_cache.json`. This prevents repeated API calls for the same executable hash during later live monitoring or manual scans. The default cache TTL is seven days.

The app respects the free public VirusTotal API rate limit by defaulting to one request every sixteen seconds. Unknown files are not uploaded unless the user explicitly changes `upload_unknown_files` in the policy.

Manual Scan Example

A single executable can be scanned directly from the command line:

```
procblart scan "C:\Program Files (x86)\Nmap\ncat.exe"
```

Example result from testing:

```
Status:      malicious
SHA-256:     c9dcde4229829484353ec43e2ae3d12a55674992db09af57af060c859b50f610
Malicious:   1
Suspicious:  0
Harmless:    0
Undetected:  71
```

> Security tools such as `ncat.exe` may receive some detections because they can be used both for legitimate administration and for abuse. The detection count should be interpreted in context rather than treated as automatic proof of malware.

4. Reaction Policy

Proc Blart evaluates each process against an editable JSON policy. The policy is intentionally easy to change because a security monitor should not require source-code edits just to adjust a threshold or response.

The default policy file is `policy.json`.

Rule	Condition	Configured Actions
`name-virus-exe`	Process name equals `virus.exe`.	`kill`, `quarantine`
`vt-detections-over-threshold`	VirusTotal malicious/suspicious detections are greater than `3`.	`suspend`, `dump_memory`, `quarantine`
`memory-over-threshold`	Resident memory usage is greater than `500 MB`.	`log_warning`

Supported actions are:

- `log_warning` : records a warning in the action log.
- `kill` : terminates the process with psutil.
- `suspend` : suspends the process with psutil.
- `dump_memory` : creates a memory dump with Sysinternals ProcDump.
- `quarantine` : moves the executable to the quarantine folder and writes metadata.

Example Policy Rule

```
{
  "id": "memory-over-threshold",
  "description": "If RSS memory is over 500 MB, log a warning.",
  "when": { "memory_mb_gt": 500 },
  "actions": ["log_warning"]
}
```

5. Dry Run and Execute Mode

The monitor starts in dry-run mode by default. In dry-run mode, Proc Blart still detects policy matches and records what would have happened, but it does not perform destructive actions.

```
procblart run -dry
```

Execute mode is explicit and intended for a controlled lab VM:

```
procblart run -exec
```

Execute mode can kill, suspend, dump, and quarantine according to the policy. The app warns if execute mode appears to be running without elevated permissions.

6. Logs and Evidence

Proc Blart writes live records as JSONL files. This makes the output easy to inspect by hand, tail in another terminal, or parse later with scripts.

Log	Purpose
`procblart_data/logs/alerts.jsonl`	Startup events, policy hits, and remote errors.
`procblart_data/logs/actions.jsonl`	Actions performed or simulated, including dry-run records.
`procblart_data/logs/virustotal.jsonl`	VirusTotal lookup results from the live monitor.

Log	Purpose
`procblart_data/logs/manual_scan.jsonl`	Manual scan support log location.

Logs can be tailed from the CLI:

```
procblart tail --log alerts
procblart tail --log actions
procblart tail --log virustotal
```

7. Remote Monitoring Extension

A personal extension was added beyond the original assignment: read-only remote monitoring for LAN machines. This makes it possible to use a large screen with several terminal panes, each watching a different host.

Supported remote modes:

- `-remote` : PowerShell CIM/WMI over WinRM/WSMan or DCOM/WMI.
- `--remote-ssh` : SSH to the remote Windows machine and run read-only PowerShell queries.

```
procblart run -remote <TARGET_IP>
procblart run --remote-ssh <USER>@<TARGET_IP>
```

Remote mode does not kill, suspend, dump, or quarantine remote processes. It collects process data, computes remote file hashes where possible, and performs VirusTotal hash lookups from the local machine.

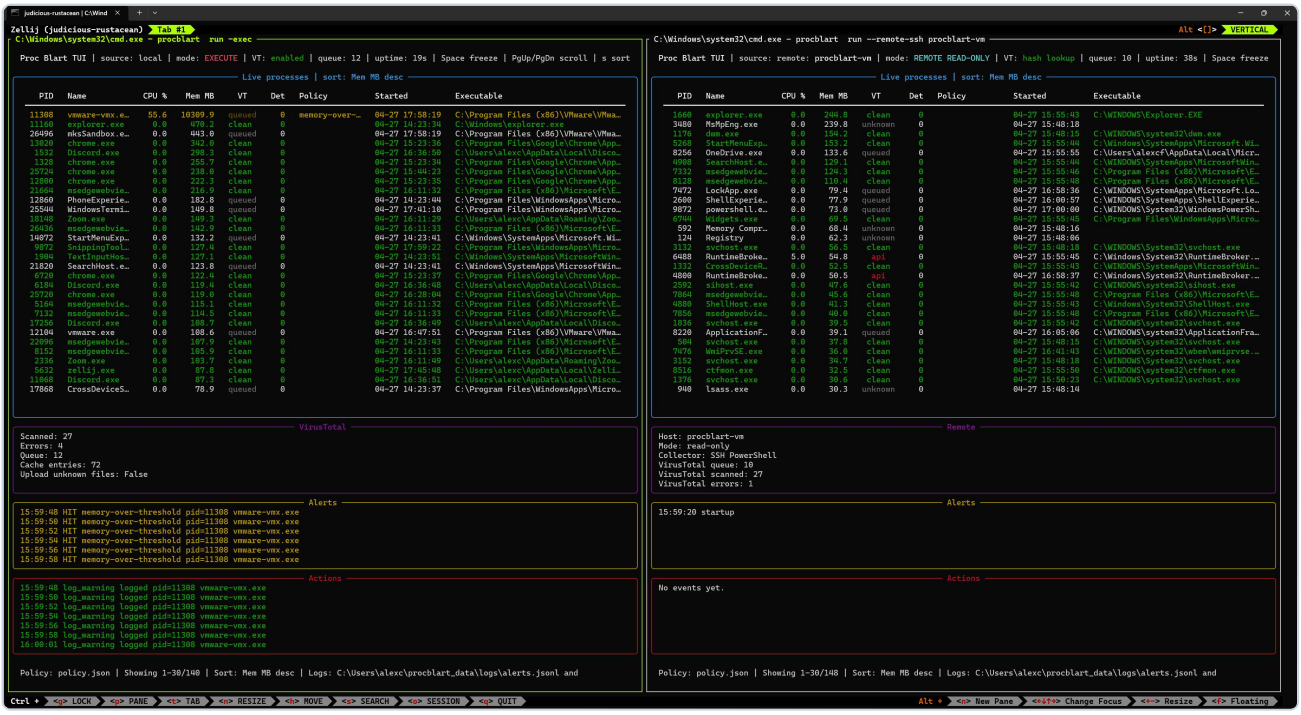

Figure 2: Proc Blart on a large screen. The layout supports a multi-pane workflow for local and remote process monitoring in parallel.

8. How the Main Components Work

Component	Responsibility
`ProcessMonitorApp`	Owns the TUI, refresh loop, keyboard controls, sorting, scrolling, and orchestration.
`psutil`	Collects local process name, PID, CPU, memory, username, executable path, and start time.
`VirusTotalClient`	Hashes executables, checks VirusTotal, rate-limits requests, and stores cache entries.
`VTScanner`	Runs VirusTotal work on a background thread so the TUI remains responsive.
`PolicyEngine`	Evaluates process rows against the JSON policy rules.
`Responder`	Performs or dry-runs actions such as kill, suspend, dump, quarantine, and warning logs.
`RemoteProcessCollector`	Collects remote Windows process data over CIM/WMI.
`SshRemoteProcessCollector`	Collects remote Windows process data and hashes through SSH PowerShell commands.

9. How to Run

Install

```
irm https://raw.githubusercontent.com/AlexCasF/procblart/main/scripts/bootstrap.ps1 | iex
```

Start the Monitor

```
procblart run -dry
```

Scan a File

```
procblart scan .\path\to\file.exe
```

Run Remote SSH Mode

```
procblart run --remote-ssh <USER>@<TARGET_IP>
```